

Harmonize EcoConnect
Unified Platform Documentation
AI-Powered, Solar Edge-Native Platform for Rural Resilience

1. Executive Summary

Harmonize EcoConnect is an AI-powered, edge-native platform engineered to strengthen rural resilience by combining predictive agriculture and primary healthcare into one intelligent, solar-powered, offline-first system.

Unlike cloud-dependent mobile apps, it operates as a standalone local cloud—a Raspberry Pi Zero 2W node that creates its own WiFi network, serves a progressive web UI to any nearby phone, and runs quantized AI models directly on-device without internet.

Key integrations:

- Appwrite – lightweight backend for authentication, user data sync, and document storage
- Sentry – local error tracking with deferred telemetry
- TailwindCSS + shadcn/ui – ultra-light, responsive UI framework
- Chart.js – micro-visualizations in analytics dashboard
- TensorFlow Lite – on-device inference for crop disease, symptom triage, and more

The platform has been submitted to UNICEF and is designed for real-world deployment in rural Nigeria: solar-powered, dust-proof, and built to survive unreliable grids and networks.

2. Problem Statement

Rural African communities face compounding barriers:

- Lack of real-time agricultural data → low yields, poor input timing, and post-harvest losses
- Fragmented health access → high-risk patients, untreated chronic conditions, and weak referral loops
- Unreliable power and network → cloud-dependent solutions fail exactly when they are needed most

Harmonize EcoConnect is built for these constraints, not despite them. Every design choice—from the solar power budget to the local WiFi access point—targets 24/7 availability in an environment where both electricity and cellular data are luxuries.

3. Solution Overview

A dual-mode, edge-first platform that delivers:

- Farm Mode – voice/keypad data logging, AI-driven crop advice, weather-informed alerts, and input-timing recommendations
- Health Mode – patient registration, symptom check against quantized clinical models, AI-generated suggestions, and medication/adherence reminders
- Financial Access – USSD/SMS-based loan requests, balance checks, and micro-credit enablement via on-device scoring
- Local Cloud – the node creates its own WiFi access point (SSID: Harmonize-Local). Any farmer or health worker can connect a basic smartphone and access the full dashboard with zero cellular data cost
- Offline-First Sync – all data is stored locally in SQLite and synced with a central Appwrite instance when a network becomes available (GSM/4G dongle or periodic WiFi).

How it works in the field:

A solar-charged node in a rural health post or cooperative office runs 24/7, providing a localized web server, AI inference, and database. Workers interact via the E-ink display on the box or via their own phones over the local WiFi. Data never leaves the community unless explicitly synced—protecting privacy and ensuring continuity even during total network blackouts.

4. Technical Architecture

4.1 Hardware Node (The “Edge-Native” Unit)

Component Specification Rationale

Processor Raspberry Pi Zero 2W (1 GHz quad-core) High performance-per-watt; tiny footprint for rugged enclosures.

Storage High-Endurance MicroSD (32 GB+, e.g., SanDisk Max Endurance) Handles heavy write loads from SQLite and Docker without card failure.

Display Waveshare 2.13” E-ink HAT (SPI) Ultra-low power, sunlight-readable; shows status even when phone is not connected.

Power Source 20 W monocrystalline solar panel + LiFePO₄ battery (12.8 V, 10 Ah) Safe, long-cycle-life battery; solar oversizing ensures charging even on cloudy days.

Charge Controller MPPT solar charge controller (e.g., Duravolt 30 A or dedicated Pi HAT) Harvests up to 30 % more energy than PWM controllers.

Voltage Regulation DC-DC buck converter (5 V, 3 A) Provides stable 5 V rail for the Pi, preventing brownout-induced SD corruption.

Enclosure IP65 weatherproof box with cable glands Protects against rain, dust, and insects.

Cooling Passive heatsink on RP3A0 chip Prevents thermal throttling during AI inference in high ambient temperatures.

RTC DS3231 real-time clock module Keeps accurate timestamps for health/agri logs across power cycles.

Estimated Prototype Cost: ~~~₦~~167,000 – ~~₦~~220,000 (USD 200–270), depending on sourcing.

4.2 Software Stack

Operating System & Base Setup

- Raspberry Pi OS Lite (64-bit), flashed headlessly with SSH pre-enabled via Raspberry Pi Imager.
- Read-only root filesystem (Overlay FS) enabled to survive sudden power cuts—logs and transient writes are stored in RAM.
- 2 GB swap file created to allow the 512 MB Pi Zero 2W to run Docker and Appwrite stably.

Local Backend & Data

- Appwrite (deployed via Docker) handles:
 - User authentication and role-based access control (RBAC)
 - Document storage for patient and farm records
 - Queue-based sync events for offline operation
 - Serverless functions for AI model updates
- SQLite serves as the primary on-device database for immediate, fast logging.
- When connectivity is available, data is synchronised with a central Appwrite instance (and, via APIs, to systems like DHIS2 or OpenMRS).

Artificial Intelligence

- TensorFlow Lite runtime runs quantized INT8 models (< 1 MB each) for:
 - Crop disease identification (e.g., cassava mosaic, brown streak)
 - Symptom triage and basic clinical decision support
 - Voice (NLP) models for USSD/speech interaction
- Inference pipeline:
 1. Image/text captured via phone UI → 2. Sent to local Flask/FastAPI endpoint on Pi → 3. TFLite interpreter returns diagnosis → 4. Result saved to Appwrite/SQLite and flashed to E-ink display.
- Power-Aware AI:
 - Battery >40 % → full high-resolution image inference
 - Battery 20–40 % → reduced inference or text-only triage
 - Battery <20 % → system enters “Sleep Mode” until solar recharge.

Frontend & User Interface

- UI framework: TailwindCSS + shadcn/ui – ultra-light components that load instantly on low-end devices.
- Analytics: Chart.js for on-dashboard micro-visualisations.
- Serving: A Python-based kiosk web view renders the UI on the E-ink display; the same UI is served over the local WiFi AP to any connected smartphone browser at <http://192.168.4.1>.
- Input methods: physical keypad, minimal touchscreen, or USSD sessions.

Networking & Communication

Protocol Use

WiFi Access Point (hostapd + dnsmasq) Local cloud – serves the web UI and receives webhook calls without internet.

GSM (USSD/SMS) Primary low-bandwidth messaging for alerts and financial transactions.

LoRa / BLE Device-to-device mesh and peripheral connections (e.g., soil sensors).

MQTT Efficient batch data sync when online; also used for OTA update signalling.

Webhook-Driven Real-Time Updates

- A lightweight FastAPI web server runs on the Pi, listening for incoming webhooks from the local Appwrite instance.
- When a new health record is created (via the mobile UI), Appwrite triggers a POST to <http://host.docker.internal:8000/webhook/new-record>.
- The Python script then updates the E-ink display immediately with the patient ID and status, providing a persistent visual log even when no phone is connected.
- This design uses internal Docker host networking to keep communication within the node's own network, adding zero latency and no external dependency.

Security & Resilience

- Appwrite RBAC for user permissions and multi-tenancy
- SQLite database encrypted at rest
- Secure Boot and signed OTA updates for the operation system and AI models
- Sentry integration: error logs are compressed and uploaded only during sync windows, preserving bandwidth
- Power management cron job: monitors battery voltage via GPIO or charge controller serial; if charge drops below 10 %, the Pi performs a clean halt to prevent filesystem corruption.

4.3 One-Click Node Provisioning

A single shell script (setup_node.sh) automates the entire software provisioning from a fresh Raspberry Pi OS Lite image:

1. System update and essential tools installation (Docker, Python, Git, hostapd, dnsmasq).
2. Swap expansion to 2 GB.

3. Appwrite deployment via the official Docker installer.
4. Python environment setup (FastAPI, TFLite runtime, Waveshare E-ink drivers).
5. SPI interface enabling.
6. Creation of a systemd service (ecoconnect.service) that auto-starts the webhook listener on every boot.

This design ensures that a “golden image” SD card can be cloned and deployed to hundreds of nodes with identical behaviour.

5. Monetization & Growth Strategy

Model Description

Device + SaaS Solar device sales coupled with subscription tiers for premium AI features, advanced analytics, and larger sync quotas.

Licensing NGOs and governments pay per farmer/patient enrolled; pricing scales with impact metrics.

Traceability-as-a-Service Opt-in supply chain logging for agricultural cooperatives (e.g., cocoa, cassava).

Microcredit Enablement On-device credit scoring; commissions earned via partner microfinance institutions when loans are disbursed.

Appwrite Workspace Multi-tenant dashboards for development partners to monitor programme performance across regions.

6. Implementation Roadmap

Q3 2025 – Platform Refinement

- Finalise hardware specification (enclosure, power circuit, pin mapping).
- Complete unified SQLite schema for health and agriculture.
- Set up USSD/SMS layer and Appwrite multi-tenancy.
- Quantize and package base AI models (crop disease, symptom triage).

Q4 2025 – Prototype & Field Feedback

- Deploy working units to pilot farmer groups and health workers in Ondo State.
- Test offline data capture, AI feedback loop, and local cloud UI.
- Gather live feedback for ergonomic and workflow improvements.

Q1 2026 – AI Expansion & Sync Engine

- Add multi-language voice support (Yoruba, Hausa, Igbo).
- Improve model precision with field data and federated fine-tuning.
- Build robust auto-sync and versioning system via Appwrite + MQTT.

7. User Guide (Conceptual Workflow)

Mode Flow

Startup Solar boot → node creates Harmonize-Local WiFi → user selects language → E-ink shows IP.

Farm Mode Farmer connects phone to WiFi → logs observations (voice/keypad) → receives AI tips and pest/disease alerts → data synced later.

Health Mode Health worker registers patient (via phone browser) → symptom check questionnaire → AI suggestion (triage category, referral urgency) → medicine reminders set.

Financial Access USSD shortcode or local web form → loan eligibility scored on-device → balance inquiry returned via SMS.

Data Sync When 4G dongle or known WiFi appears → Appwrite auto-queues records → encrypted batch upload to central dashboard.

8. Developer Notes & Quick-Start

Modular code structure: /agri, /health, /core, /models, /sync, /UI

Appwrite services used:

- Auth → user roles (farmer, health worker, admin)
- Database → document collections for patients, farms, transactions
- Functions → serverless scripts for model update and data validation
- Webhooks → real-time E-ink updates

AI model requirements:

- All models distilled, quantized, and under 1 MB
- Conversion pipeline: TensorFlow → TFLite with INT8 quantization
- Inference runtime: tflite-runtime installed via pip

CI/CD for edge OTA:

- Pipeline builds OS + software golden image
- Signed updates delivered via MQTT or USB sideload

Sentry telemetry:

- Errors captured locally, compressed, and transmitted only during scheduled sync windows to conserve battery and bandwidth.

One-command setup (summary):

```
```bash
chmod +x setup_node.sh && ./setup_node.sh
```
```

After setup, the node boots into a fully operational state: WiFi AP active, Appwrite running, webhook listener waiting, and E-ink dashboard cycling every 10 minutes.

9. Investor Positioning

Harmonize EcoConnect is:

- Edge-native – No dependency on stable internet or power grids.
- Multi-sectoral – Agriculture and healthcare in a single, low-cost device.
- Validated – Built on components already tested in rural settings (FarmConnect pilots, AI Health Navigator).
- SDG-aligned – Directly advances food security, public health, and digital inclusion (SDGs 2, 3, 9).
- Team-led – Veteran technologist with deep rural field experience in Nigeria.
- Scalable – Hardware + Appwrite backend ready to scale from 10 to 10,000+ nodes with minimal incremental cost.

Next prototype milestones (as per initial dev notes):

1. Flash the Pi Zero 2W with headless OS and verify SSH access.
2. Configure the WiFi Access Point and confirm phone connectivity.
3. Wire the solar panel, MPPT controller, battery, and verify >48 h autonomy.
4. Deploy Appwrite, run the webhook listener, and trigger an E-ink update from a test record.

This document is now a complete blueprint—ready for partners, pilots, and production.

```markdown

## # Harmonize EcoConnect — Developer Reference

\*Target audience: Developers and system integrators deploying the EcoConnect edge node.\*

This document specifies the hardware, software, and deployment process for the Harmonize EcoConnect platform.

It is a pure technical reference – for high-level mission and funding context, see the Unified Platform Documentation.

---

## ## 1. Hardware Specification

### ### 1.1 Core Components

| Component | Specification | Notes |
|-----------|---------------|-------|
|-----------|---------------|-------|

|       |       |       |
|-------|-------|-------|
| ----- | ----- | ----- |
|-------|-------|-------|

|         |                                                    |                                             |
|---------|----------------------------------------------------|---------------------------------------------|
| Compute | Raspberry Pi Zero 2W (1 GHz quad-core, 512 MB RAM) | Headless operation; no desktop environment. |
|---------|----------------------------------------------------|---------------------------------------------|

|         |                                                        |                                                 |
|---------|--------------------------------------------------------|-------------------------------------------------|
| Storage | High-Endurance microSD, 32 GB+ (SanDisk Max Endurance) | Industrial grade for SQLite/Docker write loads. |
|---------|--------------------------------------------------------|-------------------------------------------------|

|         |                                |                                            |
|---------|--------------------------------|--------------------------------------------|
| Display | Waveshare 2.13" E-ink HAT (V4) | SPI interface; ref. pinout in §Appendix A. |
|---------|--------------------------------|--------------------------------------------|

|       |                                                                     |                                                  |
|-------|---------------------------------------------------------------------|--------------------------------------------------|
| Power | 20 W mono solar panel + LiFePO <sub>4</sub> battery (12.8 V, 10 Ah) | Battery provides >48 h autonomy at typical draw. |
|-------|---------------------------------------------------------------------|--------------------------------------------------|

|                |                                                          |                               |
|----------------|----------------------------------------------------------|-------------------------------|
| Charge Control | MPPT controller (e.g., Duravolt 30 A or Pi-specific HAT) | Maximum Power Point Tracking. |
|----------------|----------------------------------------------------------|-------------------------------|

|              |                                        |                                         |
|--------------|----------------------------------------|-----------------------------------------|
| Voltage Reg. | DC-DC buck converter (5 V, 3 A output) | Feeds clean 5 V rail to Pi GPIO or USB. |
|--------------|----------------------------------------|-----------------------------------------|

|     |               |                                                   |
|-----|---------------|---------------------------------------------------|
| RTC | DS3231 module | I <sup>2</sup> C; keeps time across power cycles. |
|-----|---------------|---------------------------------------------------|

|           |                            |                                              |
|-----------|----------------------------|----------------------------------------------|
| Enclosure | IP65 box with cable glands | Passive heatsink on Pi SoC strongly advised. |
|-----------|----------------------------|----------------------------------------------|

### ### 1.2 Power Budget (typical)

- Pi Zero 2W + WiFi AP + E-ink refresh: ~150 mA @ 5 V → 0.75 W

- 10 Ah LiFePO<sub>4</sub> @ 12.8 V ≈ 128 Wh

- Autonomy: ~170 hours (7 days) without solar input; comfortably >48 h for real-world cloud cover.

---

## ## 2. System Initialisation (First Boot)

### ### 2.1 Flashing the OS



1. Use **Raspberry Pi Imager** to write `Raspberry Pi OS Lite (64-bit)` to the microSD.
2. In Imager's advanced settings:
  - Hostname: `ecoconnect.local`
  - Enable SSH (password or key)
  - Pre-configure WiFi credentials for your development network (temporary).
3. Insert SD, power on via buck converter, and SSH in:

```
```bash
ssh pi@ecoconnect.local
```
```

## 2.2 Base System Hardening

```
```bash
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-pip python3-pil python3-numpy git docker.io docker-compose
hostapd dnsmasq i2c-tools

# Expand swap to 2 GB (Appwrite/Docker survival)
sudo dphys-swapfile swapoff
sudo sed -i 's/CONF_SWAPSIZE=100/CONF_SWAPSIZE=2048/' /etc/dphys-swapfile
sudo dphys-swapfile setup
sudo dphys-swapfile swapon

# Enable SPI (E-ink) and I2C (RTC)
sudo raspi-config nonint do_spi 0
sudo raspi-config nonint do_i2c 0

# Read-only root filesystem (power-loss protection)
sudo raspi-config nonint enable_overlayfs
# After reboot, writes to / will be temporary; persistent data goes to /data or separate partition.
```
```

After enabling overlayfs, reboot. Mount any persistent writable storage under /data if needed.

---

## 3. Networking: Local Cloud Access Point

The Pi creates its own WiFi network so clients can connect without internet.

Installation:

```
```bash
```

```
sudo apt install hostapd dnsmasq -y
sudo systemctl stop hostapd dnsmasq
...

```

Static IP (/etc/dhcpd.conf append):

```
...
interface wlan0
    static ip_address=192.168.4.1/24
    nohook wpa_supplicant
...

```

DHCP range (/etc/dnsmasq.conf):

```
...
interface=wlan0
dhcp-range=192.168.4.2,192.168.4.50,255.255.255.0,24h
...

```

Access Point definition (/etc/hostapd/hostapd.conf):

```
...
country_code=NG
interface=wlan0
ssid=Harmonize-Local
hw_mode=g
channel=7
wpa=2
wpa_passphrase=YourSecurePassword
wpa_key_mgmt=WPA-PSK
...

```

Activate:

```
```bash
sudo nano /etc/default/hostapd # DAEMON_CONF="/etc/hostapd/hostapd.conf"
sudo systemctl unmask hostapd
sudo systemctl enable hostapd dnsmasq
sudo systemctl start hostapd dnsmasq
sudo reboot
...

```

Result: phones connect to Harmonize-Local and reach the UI at <http://192.168.4.1>.

---

## 4. Backend Services (Docker + Appwrite)

### 4.1 Appwrite Installation

Appwrite runs inside Docker. With limited RAM, ensure swap is active.

```
```bash
mkdir ~/appwrite && cd ~/appwrite
curl -sL https://appwrite.io/install | bash
```
```

During install, set:

- HTTP port: 80
- HTTPS port: 443 (leave disabled if no certificate)
- Hostname: localhost or 192.168.4.1

After installation, Appwrite will be available at <http://192.168.4.1>.

Access the console, create a project, and note the API Endpoint, Project ID, and API Key.

### 4.2 Docker Networking for Webhooks

To allow Appwrite containers to reach a host-running FastAPI webhook listener, add an `extra_hosts` entry in the Appwrite `docker-compose.yml` (usually in `/root/appwrite`):

```
```yaml
services:
  appwrite:
    extra_hosts:
      - "host.docker.internal:host-gateway"
```
```

Then restart: `docker-compose up -d`.

This lets Appwrite post webhooks to `http://host.docker.internal:8000/....`

---

## 5. Real-Time UI Update via Webhooks

A lightweight Python service updates the E-ink display whenever a new health record is created.

## 5.1 Webhook Listener (/home/pi/dashboard.py)

```
```python
from fastapi import FastAPI, Request
from waveshare_epd import epd2in13_V4
from PIL import Image, ImageDraw, ImageFont
import uvicorn

app = FastAPI()
epd = epd2in13_V4.EPD()

@app.post("/webhook/new-record")
async def handle_new_record(request: Request):
    payload = await request.json()
    patient_id = payload.get('$id', 'unknown')
    status = payload.get('status', 'Checkup Needed')

    try:
        epd.init()
        image = Image.new('1', (epd.height, epd.width), 255)
        draw = ImageDraw.Draw(image)
        font = ImageFont.load_default()
        draw.text((10, 10), "NEW HEALTH RECORD", font=font, fill=0)
        draw.text((10, 30), f"ID: {patient_id}", font=font, fill=0)
        draw.text((10, 50), f"STATUS: {status}", font=font, fill=0)
        epd.display(epd.getbuffer(image))
        epd.sleep()
    except Exception as e:
        print(f"Display update failed: {e}")
        raise
    return {"status": "ok"}

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```
```

## 5.2 Systemd Service

/etc/systemd/system/ecoconnect.service:

...

```
[Unit]
Description=EcoConnect Webhook Listener
After=network.target
```

```
[Service]
ExecStart=/usr/bin/python3 /home/pi/dashboard.py
WorkingDirectory=/home/pi
StandardOutput=inherit
StandardError=inherit
Restart=always
User=pi
```

```
[Install]
WantedBy=multi-user.target
...
```

Enable and start:

```
```bash
sudo systemctl enable ecoconnect.service
sudo systemctl start ecoconnect.service
```
```

### 5.3 Appwrite Webhook Configuration

In Appwrite console:

- Webhook Name: E-ink Updater
- URL: `http://host.docker.internal:8000/webhook/new-record`
- Events: `databases.[DB_ID].collections.[COLLECTION_ID].documents.create`
- Signature key: (optional) verify in Python handler.

---

## 6. AI Inference Pipeline

### 6.1 Model Preparation

All models are quantized to INT8 and kept under 1 MB. Use TensorFlow Lite Converter:

```
```python
converter = tf.lite.TFLiteConverter.from_saved_model(saved_model_dir)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.int8]
# ... representative dataset for calibration
tflite_model = converter.convert()
with open('model.tflite', 'wb') as f:
```

```
f.write(tflite_model)
...
```

Place .tflite models in /home/pi/models/.

6.2 Inference Runtime

```
```bash
pip3 install tflite-runtime
```
```

Example inference stub:

```
```python
import tflite_runtime.interpreter as tflite
import numpy as np

interpreter = tflite.Interpreter(model_path="/home/pi/models/crop_disease.tflite")
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

... preprocess image into input_data
interpreter.set_tensor(input_details[0]['index'], input_data)
interpreter.invoke()
result = interpreter.get_tensor(output_details[0]['index'])
```
```

6.3 Power-Aware Throttling

Check battery voltage before heavy inference; if below threshold, fall back to text-only models or defer.

Example voltage check via MPPT serial or GPIO voltage divider:

```
```python
Pseudocode
if get_battery_percent() < 40:
 # use simpler model or return offline message
```

---
```

7. CI/CD & OTA Updates

- Golden Image: fully configured SD card image, created once and cloned with rpi-clone or dd.
- OTA Updates: Signed update packages delivered via MQTT topic or USB sideload.
 - Package contains model files, code patches, or full OS overlays.
 - Signature verified against device-stored public key before applying.
- Sentry: error logs stored locally, compressed, and pushed during sync windows; configured via environment variables.

8. Bill of Materials (Developer Sourcing)

| # | Component | Est. Price (NGN) | Recommended Source |
|-----------------|--------------------------------|--------------------|--------------------------|
| 1 | Pi Zero 2W | 32,000 – 63,000 | Jumia, Swift Berry |
| 2 | 20W Solar Panel | 18,000 – 25,000 | FUZHAO, Qasa |
| 3 | LiFePO ₄ 12.8V 10Ah | 52,000 – 78,000 | Ubuy NG, WattCycle |
| 4 | Waveshare 2.13" E-ink | 39,000 – 48,000 | Ubuy, Waveshare official |
| 5 | MPPT Controller | 8,000 – 13,000 | Duravolt 30A |
| 6 | High-Endurance SD | 8,500 – 12,000 | SanDisk Max Endurance |
| 7 | IP65 Enclosure | 5,000 – 10,000 | Local electrical market |
| 8 | DC-DC Buck 5V 3A | 4,500 | LM2596 module |
| Total Prototype | | ~167,000 – 220,000 | |

Appendix A: E-ink Pin Mapping (Waveshare 2.13" to Pi GPIO)

| E-ink Pin | Pi GPIO | Physical Pin |
|-----------|----------------|--------------|
| VCC 3.3V | 1 or 17 | |
| GND | GND 6, 9, etc. | |
| DIN | MOSI 19 | |
| CLK | SCLK 23 | |
| CS | CE0 24 | |
| DC | GPIO25 22 | |
| RST | GPIO17 11 | |
| BUSY | GPIO24 18 | |

Appendix B: Full Setup Script

Combine all steps into a single provisioning script (setup_node.sh). Refer to the earlier merged documentation for the complete script; run it on a fresh Pi OS Lite to go from zero to operational node in ~30-45 minutes.

Document version 2.0 – Developer edition. Last updated: 2026-05-06.
